

LLMOrchestrator

LLMOrchestrator

प्रत्येक हेडलेस CLI कोडिंग एजेंट के लिए एक नियंत्रण प्लेन।

सारांश

LLMOrchestrator एक स्वतंत्र, पुनःप्रयोगी Go मॉड्यूल है, जो हाइब्रिड पाइप+फाइल प्रोटोकॉल के माध्यम से हेडलेस CLI एजेंट्स (OpenCode, Claude Code, Gemini, Junie, Qwen Code) को उत्पन्न, प्रबंधित और उनसे संवाद करने के लिए है। इसमें प्रति-एजेंट सर्किट ब्रेकर, प्लग करने योग्य मल्टी-प्रोवाइडर चयन, एक अलग किया गया i18n एब्स्ट्रैक्शन और एंटी-ब्लफ़ टेस्ट गारंटी शामिल हैं।

संक्षिप्त विवरण

एक पुनःप्रयोगी Go मॉड्यूल जो हाइब्रिड पाइप+फाइल प्रोटोकॉल के माध्यम से कई LLM-संचालित CLI एजेंट्स को उत्पन्न और संचालित करने के लिए एकीकृत इंटरफ़ेस प्रदान करता है। थ्रेड-सुरक्षित एजेंट पूलिंग के साथ सर्किट ब्रेकर और चुनिंदा रूटिंग रणनीतियाँ, जानबूझकर उपभोक्ता-अज्ञेयवादी, प्लग करने योग्य i18n अनुवादक के साथ।

विस्तृत विवरण

LLMOrchestrator हेडलेस CLI कोडिंग एजेंट्स के ऑर्किस्ट्रेशन के लिए साझा इंफ्रास्ट्रक्चर है — वह आधारभूत ढाँचा जो हर मल्टी-एजेंट प्रणाली चुपचाप चाहती है और आमतौर पर खराब तरीके से फिर से बनाती है। OpenCode, Claude Code, Gemini CLI, Junie और Qwen Code जैसे टूल्स के लिए हर प्रोजेक्ट द्वारा प्रक्रिया उत्पन्न करना, संदेश फ्रेमिंग और परिणाम पार्सिंग को फिर से लागू करने के बजाय, यह एक एकीकृत Agent इंटरफ़ेस, एक थ्रेड-सुरक्षित AgentPool और एक MultiProviderPool प्रदान करता है, जो कई प्रोवाइडर्स से एजेंट्स को एक ही फ़ेसाड के पीछे प्रबंधित करता है। रूटिंग एक AgentSelector के माध्यम से प्लग करने योग्य है — राउंड-रॉबिन जो आवश्यकताओं को पूरा न करने वाले प्रोवाइडर्स को छोड़ देता है, या प्राथमिकता-क्रम में फ़ॉलबैक के साथ — ताकि कार्य वितरण की नीति आप चुनें, न कि कोई हार्डकोडेड धारणा। प्रत्येक ठोस एजेंट एक साझा BaseAdapter पर आधारित पतला एडाप्टर होता है, जो पूरी प्रक्रिया के जीवनचक्र का स्वामित्व रखता है: पाइप सेटअप के साथ शुरू करना, ग्रेसफुल SIGTERM-फिर-SIGKILL स्टॉप, पुनः आरंभ और जीवंतता — वह जटिल और त्रुटि-प्रवण हिस्सा, जिसे एक बार हल कर दिया गया है।

संचार जानबूझकर हाइब्रिड है, जो कार्य के अनुरूप परिवहन माध्यम का चयन करता है। एक पाइप ट्रांसपोर्ट न्यूलाइन-डिलिमिटेड JSON को प्रति-अनुरोध रीड डेडलाइन और प्रतिक्रिया-लंबाई सीमा के साथ तेज़ इंटरैक्टिव संदेशों के लिए ले जाता है, जबकि फ़ाइल ट्रांसपोर्ट बड़े या टिकाऊ आर्टिफैक्ट्स के लिए प्रति-सत्र इनबॉक्स/आउटबॉक्स/साझा निर्देशिकाओं का उपयोग करता है, जिन्हें पाइप में नहीं रखा जाना चाहिए। लचीलापन कोई बाद का विचार नहीं है — यह संरचनात्मक है: प्रति-एजेंट सर्किट ब्रेकर तीन लगातार विफलताओं के बाद 60 सेकंड के कूलडाउन के लिए खुलता है, जिसके बाद एक हाफ-ओपन प्रोब होता है, और एक बैकग्राउंड हेल्थ मॉनिटर एजेंट्स को पिंग करता है ताकि डाउन हुए एजेंट को आने वाले ट्रैफ़िक का इंतज़ार किए बिना पुनर्प्राप्त किया जा सके। पूल अधिग्रहण एक कंडीशन वेरिएबल पर ब्लॉक करता है, CPU को व्यस्त-प्रतीक्षा में जलाने के बजाय, और प्रतिक्रिया पार्सर स्टेटलेस और समवर्ती कॉल के लिए सुरक्षित है। मॉड्यूल पूरी तरह से अलग किया गया है — किसी भी उपभोक्ता-विशिष्टता को अंदर आने की अनुमति नहीं है — और हर उपयोगकर्ता-सामना करने वाला स्ट्रिंग एक प्लग करने योग्य i18n Translator से गुजरता है, जिसमें एक NoopTranslator होता है जो संदेश आईडी को ज्यों का त्यों लौटाता है ताकि अनुपलब्ध अनुवाद स्पष्ट रूप से दिखाई दे, छिपे नहीं।

हमने इसे क्यों बनाया

हर मल्टी-एजेंट प्रणाली को CLI एजेंट्स को विश्वसनीय रूप से लॉन्च और संचालित करने की आवश्यकता होती है। हर प्रोजेक्ट में स्पॉनिंग, फ्रेमिंग, पार्सिंग और विफलता-प्रबंधन को फिर से हल करना बेकार और त्रुटि-प्रवण है। LLMOrchestrator इसे एक अलग, पुनःप्रयोगी मॉड्यूल में केंद्रीकृत करता है, जिसकी विशेष जिम्मेदारी इसे पुनःप्रयोगी बनाती है — और वह पुनःप्रयोगिता तब नष्ट हो जाती है जब किसी उपभोक्ता की विशिष्टताएँ अंदर आ जाती हैं।

सामग्री

यह क्यों एक गेम-चेंजर है

यह “विभिन्न प्रकार के CLI एजेंटों की सेना चलाने” को एक कस्टम प्रोजेक्ट-विशिष्ट इंजीनियरिंग संघर्ष से बदलकर एकल लाइब्रेरी आयात में तब्दील कर देता है — पूलिंग, सर्किट ब्रेकिंग, लाइफसाइकिल प्रबंधन और प्लगएबल रूटिंग पहले से ही हल और मजबूत की गई हैं। और चूँकि इसके एंटी-ब्लफ़ टेस्ट वास्तविक सिस्टम को एंड-टू-एंड टेस्ट करते हैं, न कि केवल “यह कंपाइल होता है” पर निर्भर रहते हैं, इसलिए आपको एक ऐसा एब्स्ट्रैक्शन मिलता है जिस पर आप वास्तविक कंकरेंसी और विफलता की स्थितियों में भरोसा कर सकते हैं, न कि केवल एक ऐसा जो डायग्राम में सही दिखता हो।

क्या है नवाचारी

- हाइब्रिड पाइप+फाइल प्रोटोकॉल — इंटरैक्टिव गति (JSON-लाइनों के माध्यम से stdin/stdout, रीड डेडलाइन, रिस्पॉन्स कैप) और बड़े आर्टिफैक्ट्स के लिए टिकाऊ फाइल-आधारित आदान-प्रदान (इनबॉक्स/आउटबॉक्स/शेयर्ड), ताकि आप कभी भी लेटेंसी और टिकाऊपन के बीच समझौता न करना पड़े।
- मल्टी-प्रोवाइडर पूल विथ प्लगएबल सिलेक्टर्स — कई CLI प्रोवाइडर्स पर एक ही फेसाड, जिसमें राउंड-रॉबिन या प्राथमिकता-आधारित रूटिंग को नीतिगत रूप से चुना जाता है, न कि हार्डकोड किया हुआ।

- **प्रति-एजेंट सर्किट ब्रेकर + बैकग्राउंड हेल्थ मॉनिटर** — स्वचालित डिग्रेडेशन और रिकवरी (3 विफलताएँ → 60 सेकंड ओपन → हाफ-ओपन प्रोब), ताकि एक अस्थिर एजेंट को अलग किया जा सके और फिर बिना मैनुअल हस्तक्षेप के चुपचाप वापस लाया जा सके।
- **नॉन-बिजी-वेट पूलिंग** — Acquire तब तक sync.Cond पर ब्लॉक रहता है जब तक कोई मिलान वाला, स्वस्थ एजेंट मुक्त नहीं हो जाता या कॉन्टेक्स्ट रद्द नहीं हो जाता, ताकि प्रतीक्षा में सीपीयू का कोई खर्च न हो।
- **सख्त डिकपलिंग + एंटी-ब्लफ़ i18n** — NoopTranslator संदेश आईडी को ज्यों का त्यों लौटाता है, ताकि अनुवाद की कमी को नज़रअंदाज़ करना असंभव हो, बजाय चुपचाप खाली छोड़ने के।
- **सुरक्षा-डिफ़ॉल्ट** — बाइनरी-पथ की अनुमति सूची का मतलब है कोई शेल इंटरपोलेशन नहीं और इसलिए कोई कमांड-इंजेक्शन सतह नहीं, साथ ही पथ-ट्रैवर्सल सुरक्षा, 1 MiB की रिस्पॉन्स कैप अनियंत्रित आउटपुट के खिलाफ़, और लॉग्स में API-कुंजी मास्किंग।
- **एंटी-ब्लफ़ चैलेंज हार्नेस** — पाँच भाषाओं (en/sr/ja/es/de) में वास्तविक डिस्क/JSON/पार्सर राउंड-ट्रिप्स, जिसमें एक म्यूटेशन गेट भी शामिल है जो फीचर के टूटने पर नॉन-ज़ीरो एग्जिट कोड देता है — एक ऐसा टेस्ट जो साबित करता है कि यह वास्तव में विफल हो सकता है।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **विश्वसनीय एजेंट प्रक्रिया I/O** — एक स्पॉन किए गए CLI प्रक्रिया से संवाद करना आश्चर्यजनक रूप से कठिन है; इसे हाइब्रिड पाइप+फाइल ट्रांसपोर्ट, एक परिभाषित संदेश/पार्सर अनुबंध (ताकि दोनों पक्ष वायर फॉर्मेट पर सहमत हों), और एक BaseAdapter से हल किया गया जो पूरी प्रक्रिया के लाइफसाइकिल को केंद्रीकृत करता है, जिसमें एक ग्रेसफुल SIGTERM टाइमआउट भी शामिल है जो SIGKILL फॉलबैक तक बढ़ता है।
- **बिजी-वेटिंग के बिना कंकरेंसी** — इसे म्यूटेक्स + कंडीशन-वेरिएबल AgentPool से हल किया गया, जहाँ Acquire तब तक स्लीप मोड में रहता है जब तक कोई क्षमता-मिलान वाला एजेंट वास्तव में मुक्त नहीं हो जाता, साथ ही एक स्टेटलेस, साइड-इफ़ेक्ट-फ्री पार्सर जो कई गो-रूटीन से एक साथ कॉल करने के लिए सुरक्षित है।
- **प्रोवाइडर विफलता अलगाव** — इसे इस तरह हल किया गया कि एक खराब प्रोवाइडर बाकी को प्रभावित न कर सके प्रति-एजेंट सर्किट ब्रेकर विस्फोट त्रिज्या को सीमित करते हैं, और एक हेल्थ-मॉनिटर गो-रूटीन रिकवरी को संचालित करता है, भले ही कोई अनुरोध न आए जो इसे ट्रिगर करे।
- **सिर्फ कंपाइलेशन नहीं, बल्कि शुद्धता का प्रमाण** — इसे चैलेंज रनर से हल किया गया: en/sr/ja/es/de में दर्जनों इन्वेरिएंट्स जो वास्तविक सिस्टम का परीक्षण करते हैं, साथ ही एक म्यूटेशन गेट (LLMORCH_MUTATE_RUNNER=1 को विफल होना चाहिए → रैपर एग्जिट 99) जो जानबूझकर फीचर को तोड़ता है ताकि यह साबित हो सके कि गेट खुद एक ब्लफ़ नहीं है।
- **स्थानीयकरण बिना चुपचाप विफलता के** — इसे वबैटिम-आईडी NoopTranslator सीम और प्रति-उपभोक्ता ट्रांसलेटर इंजेक्शन से हल किया गया, ताकि अनुवाद में कोई कमी हमेशा दिखाई दे, बजाय उसे छिपाने के।

सामग्री

तकनीकी स्टैक

- **Go (1.25)** — उच्च-स्तरीय समवर्तीता और स्वच्छ प्रक्रिया नियंत्रण के लिए चुना गया, जो लाइव एजेंट प्रक्रियाओं के ऑर्केस्ट्रेशन की सटीक माँग करता है; यह मॉड्यूल, इसके एजेंट एडेप्टर, ट्रांसपोर्ट और पार्सर को लागू करता है।
- **Go stdlib मात्र (+ testify, yaml.v3)** — निर्भरताओं की सतह को न्यूनतम रखने और कोई LLM SDK न जोड़ने का जानबूझकर किया गया चुनाव, ताकि मॉड्यूल हल्का रहे और किसी भी उपभोक्ता के भीतर एम्बेड किया जा सके बिना विक्रेता-विशिष्ट बोझ के।
- **पाइप ट्रांसपोर्ट (JSON-lines stdio पर आधारित)** — तेज़ इंटरैक्टिव संदेशवाहन के लिए चुना गया, जिसे पढ़ने की समय-सीमा और प्रतिक्रिया-लंबाई की सीमाओं से मजबूत किया गया है ताकि अटक या अनियंत्रित एजेंट कॉलर को रोक न सके।
- **फ़ाइल ट्रांसपोर्ट (इनबॉक्स/आउटबॉक्स/साझा)** — सत्र-आधारित टिकाऊ, बड़े आर्टिफैक्ट विनिमय के लिए चुना गया, जहाँ पाइप गलत उपकरण साबित होता।
- **sync.Mutex/sync.Cond** — बिना व्यस्त-प्रतीक्षा के निष्पक्ष एजेंट-पूल अधिग्रहण लागू करने के लिए चुना गया।
- **सर्किट ब्रेकर + हेल्थ मॉनिटर** — एक साथ चुने गए ताकि प्रति-एजेंट लचीलापन और सक्रिय पुनर्प्राप्ति सुनिश्चित हो, न कि केवल विफलता का पता लगाना।
- **pkg/i18n अनुवादक** — मुख्य कोड से उपभोक्ता-विशिष्ट स्ट्रिंग्स को अलग रखने वाले स्थानीयकरण के लिए चुना गया।
- **चैलेंज हार्नेस (challenges/runner) + Makefile (test -race, fuzz, cover)** — धोखाधड़ी-रोधी, साक्ष्य-आधारित सत्यापन के लिए चुना गया, जिसमें रेस डिटेक्शन और पार्सर फ़ज़िंग शामिल है ताकि प्रतिकूल परिस्थितियों में भी शुद्धता सिद्ध हो, न कि केवल मान ली जाए।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। एक अलग पुनःप्रयोगी मॉड्यूल, जिसे कई Helix/vasic परियोजनाओं द्वारा सबमॉड्यूल के रूप में उपयोग किया जाता है। **लाइसेंस:** **Apache-2.0**; GitHub रिपॉजिटरी सार्वजनिक है।
- मॉडल मेटाडेटा LLMsVerifier से HelixQA के माध्यम से प्राप्त होता है; यह मॉड्यूल सीधे LLMsVerifier/VisionEngine/DocProcessor को आयात नहीं करता। पैरेंट ऐप के CLAUDE.md (Gin/PostgreSQL आदि) में उल्लिखित स्टैक `helix_code` का वर्णन करते हैं, इस मॉड्यूल का नहीं।

प्राथमिकता स्तर: Helix-प्राथमिक (LLM-इन्फ्रास्ट्रक्चर क्लस्टर — अलग पुनःप्रयोगी मॉड्यूल)। HelixTrack के बाद आता है।